Caden Sun

+1 647-640-2255 | cadensun07@gmail.com | github.com/Quaden2307 | linkedin.com/in/cadensun

Skills

Languages: Python, JavaScript/TypeScript, Java, C/C++, SQL, HTML/CSS
Frameworks: PyTorch, Pandas, NumPy, Matplotlib, Scikit-learn, XGBoost, Flask, React, Node.js, LangGraph
Tools & Platforms: AWS (S3, EC2, DynamoDB), Docker, CI/CD, Kubernetes, Linux, Git, Vercel

Education

University of Waterloo Sep 2025 – Apr 2030
Bachelor of Mathematics
Relevant Coursework: Data Structures & Algorithms, Designing Functional Programs, Linear Algebra, Probability & Statistics

Experience

AI Intern | *ProCogia* May 2026 – Present
- Engineered an **open-source voice AI agent** POC in under a week, migrating from **Amazon Connect** to **LiveKit**, **Moonshine STT**, and a **self-hosted Qwen-32B LLM** on EC2, which reduced **vendor lock-in** and replaced usage-based cloud costs with a fixed compute footprint.
- Reduced **latency to under 500 ms** (a **3× improvement** over the initial prototype) by re-architecting the agent pipeline across **LangGraph** and **Twilio** with a modular, **distributed** design.
- Improved **RAG workflow** retrieval accuracy by designing and deploying **production-grade API integrations** and cloud infrastructure on **AWS** (**S3, EC2, DynamoDB**), enabling scalable document ingestion for client workloads.
- Built an **automated smoke test suite** for a production **AWS** pipeline: validated **Lambda** health, **SQS** queue state, **DLQ** depth, and **DynamoDB** claim seeding in a single script.

ML Engineer | *Wat Street (University of Waterloo's Quantitative Finance Team)* Jan 2026 – Present
- Engineered a scalable **data pipeline** (**Python**, **Pandas**, **NumPy**) ingesting live **S&P 500** market data and producing structured **tensor inputs** across **500 equities** per batch, supporting volatility forecasting models.
- Researched and evaluated ML architectures for multi-stock volatility forecasting, surveying **HAR-RV**, **GARCH**, **LSTM**, and **Graph Attention Network (GAT)** approaches to inform model selection for a **Temporal GNN** system.
- Improved **pipeline reliability** by writing **unit tests** for pipeline stages and debugging data integrity issues, catching **data leakage bugs** before model evaluation.

Software Developer | *Waterloo Aerial Robotics Group (WARG)* Oct 2025 – Apr 2026
- Increased ground control station UI by implementing a **coordinate mapping system** in **JavaScript** for **IMACS-3.0**, integrating it into the existing **backend architecture** and validating against live **flight telemetry**.
- Cut field debugging time by building a **real-time flight monitoring dashboard** in **Dart/Flutter**, displaying live telemetry and alerts across **Android and desktop** using **OOP** patterns for reusable, testable components.

Projects

Flight Price Predictor | *Python, SQLite, XGBoost, Pandas, NumPy, Scikit-learn* |
- Building an **end-to-end flight price prediction system** covering **200+ routes** across **55+ airports** — training an **XGBoost gradient-boosted model** on engineered features (route distance, lead time, day-of-week, airline type, transfer count) informed by **18 plots of documented EDA** across two data phases, targeting **sub-10% MAPE**.
- Built a **fault-tolerant ingestion pipeline** (**Python** + **SQLite** + **launchd**) pulling **∼4,000 flight offers/day** during a planned **four-month** collection period — reduced **final failure rate to 0%** across multiple slow-API days, with **automated cloud backup** and **per-run logging**.
- Maintained **dataset quality** by authoring automated **SQL deduplication** and **integrity-check scripts** running daily after ingestion — **9-column exact-key dedup guard**, **NULL/range audits** on modeling-critical fields, and **volume-drop detection** against a 3-day rolling baseline; **0 detected anomalies** across **60K+ rows** to date.

AI Chess Bot | *Python, PyTorch, Flask, React/TypeScript, Docker* | |
- Deployed a **full-stack chess AI** on **Vercel** (containerized with **Docker** for an alternative **Render** deployment), serving a **PyTorch** neural network (**768 → 256 → 128 → 1**, **ReLU**) via **Flask REST API** and **React/TypeScript** frontend — **sub-second response times**, publicly accessible.
- Improved **model generalization** (**30% reduction in validation loss**) across **4 training iterations** from a baseline evaluator to a tactical model using **Stockfish**-labeled data, **Adam** optimizer, **dropout**, **batch normalization**, and hand-crafted signals: material balance, castling rights, and center control.